Assignment 2 — Solutions

Christopher Elliott
A01526751

Throughout, log means $\log_2$, following the convention of [1, p. 22].

2.1

Let $x = 10011011_2 = 155$ and $y = 10111010_2 = 186$. I apply the algorithm of Figure 2.1 of [1]. On inputs whose larger size is n bits, it sets $h = \lfloor n/2 \rfloor$, splits $x = 2^h x_L + x_R$ and $y = 2^h y_L + y_R$ (so x_R, y_R are the rightmost h bits and x_L, y_L the remaining leftmost $\lceil n/2 \rceil$ bits), computes the three recursive products

$$P_1 = x_L y_L, \qquad P_2 = x_R y_R, \qquad P_3 = (x_L + x_R)(y_L + y_R),$$

and returns

$$xy = P_1 \cdot 2^{2h} + (P_3 - P_1 - P_2) \cdot 2^h + P_2.$$

This identity holds because $xy = 2^{2h} x_L y_L + 2^h (x_L y_R + x_R y_L) + x_R y_R$ and $x_L y_R + x_R y_L = (x_L + x_R)(y_L + y_R) - x_L y_L - x_R y_R$. (For even n, $2^{2h} = 2^n$ and $2^h = 2^{n/2}$, exactly as written in Figure 2.1. The sums $x_L + x_R$ and $y_L + y_R$ can be one bit longer than h, so some recursive calls have odd n.) The base case $n = 1$ returns the product of two bits directly.

IDEA

Split each 8-bit number into two 4-bit halves, replace the four half-size products by three (Gauss's trick), and recurse on each of the three until the numbers are single bits. Then put the pieces back together with shifts and additions.

Top level ($n = 8$, $h = 4$).

$$x_L = 1001_2 = 9, \qquad x_R = 1011_2 = 11, \qquad x_L + x_R = 10100_2 = 20,$$
$$y_L = 1011_2 = 11, \qquad y_R = 1010_2 = 10, \qquad y_L + y_R = 10101_2 = 21.$$

So $P_1 = 9 \cdot 11$, $P_2 = 11 \cdot 10$ and $P_3 = 20 \cdot 21$, each computed recursively below.

Second level. Each row is one recursive call, split the same way.

Call	n	h	x_L	x_R	y_L	y_R	P_1	P_2	P_3	Result
$9 \cdot 11 = 1001_2 \cdot 1011_2$	4	2	10_2	01_2	10_2	11_2	$2 \cdot 2$	$1 \cdot 3$	$3 \cdot 5$	99
$11 \cdot 10 = 1011_2 \cdot 1010_2$	4	2	10_2	11_2	10_2	10_2	$2 \cdot 2$	$3 \cdot 2$	$5 \cdot 4$	110
$20 \cdot 21 = 10100_2 \cdot 10101_2$	5	2	101_2	00_2	101_2	01_2	$5 \cdot 5$	$0 \cdot 1$	$5 \cdot 6$	420

Third level. The products of 3-bit numbers recurse once more with $h = 1$:

Call	n	h	x_L	x_R	y_L	y_R	P_1	P_2	P_3	Result
$3 \cdot 5 = 011_2 \cdot 101_2$	3	1	1_2	1_2	10_2	1_2	$1 \cdot 2 = 2$	$1 \cdot 1 = 1$	$2 \cdot 3 = 6$	$2 \cdot 4 + 3 \cdot 2 + 1 = 15$
$5 \cdot 4 = 101_2 \cdot 100_2$	3	1	10_2	1_2	10_2	0_2	$2 \cdot 2 = 4$	$1 \cdot 0 = 0$	$3 \cdot 2 = 6$	$4 \cdot 4 + 2 \cdot 2 + 0 = 20$
$5 \cdot 5 = 101_2 \cdot 101_2$	3	1	10_2	1_2	10_2	1_2	$2 \cdot 2 = 4$	$1 \cdot 1 = 1$	$3 \cdot 3 = 9$	$4 \cdot 4 + 4 \cdot 2 + 1 = 25$
$5 \cdot 6 = 101_2 \cdot 110_2$	3	1	10_2	1_2	11_2	0_2	$2 \cdot 3 = 6$	$1 \cdot 0 = 0$	$3 \cdot 3 = 9$	$6 \cdot 4 + 3 \cdot 2 + 0 = 30$

The remaining calls are on numbers of at most 2 bits. A 2-bit call has $h = 1$ and three 1-bit subproducts. For example, $3{\cdot}3 = 11_2{\cdot}11_2$ gives $P_1 = 1{\cdot}1 = 1$, $P_2 = 1{\cdot}1 = 1$, $P_3 = (1{+}1)(1{+}1) = 2{\cdot}2$; the last is again a 2-bit call, giving $P_1 = 1$, $P_2 = 0$, $P_3 = 1$ and so $2 \cdot 2 = 1 \cdot 4 + 0 \cdot 2 + 0 = 4$. Then $3 \cdot 3 = 1 \cdot 4 + (4 - 1 - 1) \cdot 2 + 1 = 9$. The other 2-bit calls are evaluated the same way and give $2 \cdot 2 = 4$, $1 \cdot 3 = 3$, $1 \cdot 2 = 2$, $2 \cdot 3 = 6$ and $3 \cdot 2 = 6$.

Combining the second level.

$$\begin{aligned} 9 \cdot 11 &= 4 \cdot 2^4 + (15 - 4 - 3) \cdot 2^2 + 3 = 64 + 32 + 3 = 99, \\ 11 \cdot 10 &= 4 \cdot 2^4 + (20 - 4 - 6) \cdot 2^2 + 6 = 64 + 40 + 6 = 110, \\ 20 \cdot 21 &= 25 \cdot 2^4 + (30 - 25 - 0) \cdot 2^2 + 0 = 400 + 20 + 0 = 420. \end{aligned}$$

Combining the top level. $P_1 = 99 = 1100011_2$, $P_2 = 110 = 1101110_2$, $P_3 = 420 = 110100100_2$, so $P_3 - P_1 - P_2 = 211 = 11010011_2$, and

$$\begin{array}{rl} \texttt{110001100000000} & (P_1 \cdot 2^8) \\ +\,\texttt{000110100110000} & ((P_3 - P_1 - P_2) \cdot 2^4) \\ +\,\texttt{000000001101110} & (P_2) \\ \hline \texttt{111000010011110} & \end{array}$$

that is, $25344 + 3376 + 110 = 28830$. Therefore

$$10011011_2 \times 10111010_2 = 111000010011110_2 \ (= 28830).$$

2.3

Throughout, as in the exercise, I read $O(n)$ as "at most cn" and $O(1)$ as "at most c" for a fixed constant $c > 0$, and assume $T(1)$ is a constant. In part (a) n is a power of 2, so every $n/2^k$ is an integer.

IDEA

Unroll the recurrence a few times, guess the general kth line, prove the guess by induction on k, and then choose k so that the remaining $T(\cdot)$ term is the known base case $T(1)$.

(a) $T(n) \le 3T(n/2) + cn$. Expanding,

$$\begin{aligned} T(n) &\le 3T(n/2) + cn \\ &\le 3[3T(n/4) + cn/2] + cn = 9T(n/4) + cn\left(1 + \tfrac{3}{2}\right) \\ &\le 9[3T(n/8) + cn/4] + cn\left(1 + \tfrac{3}{2}\right) = 27T(n/8) + cn\left(1 + \tfrac{3}{2} + \tfrac{9}{4}\right). \end{aligned}$$

Each line multiplies the coefficient of T by 3 and adds the next term of the geometric series with ratio $3/2$. The general kth term is

$$T(n) \le 3^k T(n/2^k) + cn \sum_{i=0}^{k-1} \left(\tfrac{3}{2}\right)^i = 3^k T(n/2^k) + 2cn \left[\left(\tfrac{3}{2}\right)^k - 1\right],$$

using $\sum_{i=0}^{k-1} r^i = (r^k - 1)/(r - 1)$ with $r = 3/2$.

Proof of the general term by induction on k, for $0 \le k \le \log_2 n$. For $k = 0$ it reads $T(n) \le T(n) + 0$. Suppose it holds for some $k < \log_2 n$. Applying the recurrence to $n/2^k \ge 2$ gives $T(n/2^k) \le 3T(n/2^{k+1}) + cn/2^k$, so

$$\begin{aligned} T(n) &\le 3^k \left[3T(n/2^{k+1}) + \frac{cn}{2^k}\right] + 2cn \left[\left(\tfrac{3}{2}\right)^k - 1\right] \\ &= 3^{k+1} T(n/2^{k+1}) + cn \left[\left(\tfrac{3}{2}\right)^k + 2\left(\tfrac{3}{2}\right)^k - 2\right] = 3^{k+1} T(n/2^{k+1}) + 2cn \left[\left(\tfrac{3}{2}\right)^{k+1} - 1\right], \end{aligned}$$

since $3\left(\tfrac{3}{2}\right)^k = 2\left(\tfrac{3}{2}\right)^{k+1}$. This is the statement for $k + 1$.

Value of k. The base case is reached when $n/2^k = 1$, so plug in $k = \log_2 n$. Then $3^k = 3^{\log_2 n} = n^{\log_2 3}$ (both equal $2^{(\log_2 3)(\log_2 n)}$), and $(3/2)^k = 3^{\log_2 n}/2^{\log_2 n} = n^{\log_2 3}/n$. Hence

$$T(n) \le n^{\log_2 3} T(1) + 2cn \left[\frac{n^{\log_2 3}}{n} - 1\right] = \big(T(1) + 2c\big) n^{\log_2 3} - 2cn \le \big(T(1) + 2c\big) n^{\log_2 3},$$

so $T(n) = O(n^{\log_2 3}) \approx O(n^{1.59})$.

(b) $T(n) \le T(n - 1) + c$. The master theorem does not apply, because the subproblem has size $n - 1$ rather than n/b for a constant $b > 1$. Expanding,

$$\begin{aligned} T(n) &\le T(n-1) + c \\ &\le [T(n-2) + c] + c = T(n-2) + 2c \\ &\le [T(n-3) + c] + 2c = T(n-3) + 3c, \end{aligned}$$

and the general kth term is

$$T(n) \le T(n - k) + kc, \qquad 0 \le k \le n - 1.$$

Proof by induction on k: for $k = 0$ it is $T(n) \le T(n)$. If it holds for $k < n - 1$, then $n - k \ge 2$, so $T(n - k) \le T(n - k - 1) + c$ and $T(n) \le T(n - (k + 1)) + (k + 1)c$.

Plugging in $k = n - 1$ reaches the base case:

$$T(n) \le T(1) + (n - 1)c \le \big(T(1) + c\big) n,$$

so $T(n) = O(n)$.

2.4

Let $c > 0$ be a constant bounding the combining cost of each algorithm (for example cn for A), and let the running times on constant-size inputs be constants. I use the master theorem of Section 2.2

of [1]: if $T(n) = aT(\lceil n/b \rceil) + O(n^d)$ with $a > 0$, $b > 1$, $d \geq 0$, then $T(n) = O(n^d)$ if $d > \log_b a$, $O(n^d \log n)$ if $d = \log_b a$, and $O(n^{\log_b a})$ if $d < \log_b a$.

IDEA

Write a recurrence for each algorithm. A and C fit the master theorem; B does not (its subproblems shrink by 1, not by a factor), so unroll it as in Exercise 2.3(b). Then compare the three growth rates.

Algorithm A. Five subproblems of half the size, plus linear combining:

$$T_A(n) = 5T_A(n/2) + O(n).$$

Here $a = 5$, $b = 2$, $d = 1$, and $\log_2 5 \approx 2.32 > 1 = d$, so the third case gives

$$T_A(n) = O(n^{\log_2 5}) \approx O(n^{2.32}).$$

Algorithm B. Two subproblems of size $n - 1$, plus constant combining:

$$T_B(n) \leq 2T_B(n-1) + c.$$

Unrolling gives $T_B(n) \leq 4T_B(n-2) + 3c \leq 8T_B(n-3) + 7c$, and in general

$$T_B(n) \leq 2^k T_B(n-k) + c\,(2^k - 1), \qquad 0 \leq k \leq n-1.$$

Proof by induction on k: $k = 0$ is trivial. If it holds for $k < n-1$, then $T_B(n-k) \leq 2T_B(n-k-1)+c$, so $T_B(n) \leq 2^{k+1}T_B(n-k-1) + 2^k c + c(2^k - 1) = 2^{k+1}T_B(n-(k+1)) + c(2^{k+1} - 1)$.

With $k = n - 1$:

$$T_B(n) \leq 2^{n-1}T_B(1) + c\,(2^{n-1} - 1) \leq \big(T_B(1) + c\big)2^{n-1},$$

so $T_B(n) = O(2^n)$.

Algorithm C. Nine subproblems of a third of the size, plus quadratic combining:

$$T_C(n) = 9T_C(n/3) + O(n^2).$$

Here $a = 9$, $b = 3$, $d = 2$, and $\log_3 9 = 2 = d$, so the second case gives

$$T_C(n) = O(n^2 \log n).$$

Which to choose. The bounds above are also tight when each combining step really takes time proportional to n, 1 and n^2 respectively (the master theorem then holds with Θ, and for B the unrolled recurrence holds with equality, giving $\Theta(2^n)$). So it is enough to compare the three functions.

- C versus A: let $\varepsilon = \log_2 5 - 2 > 0$. Then

  $$\frac{n^2 \log n}{n^{\log_2 5}} = \frac{\log n}{n^{\varepsilon}} \to 0,$$

  because by L'Hôpital's rule $\dfrac{\ln n}{n^{\varepsilon}}$ has the same limit as $\dfrac{1/n}{\varepsilon n^{\varepsilon - 1}} = \dfrac{1}{\varepsilon n^{\varepsilon}} \to 0$ (changing the base of the log only multiplies by a constant). So $n^2 \log n$ grows strictly slower than $n^{\log_2 5}$.

- A versus B: any polynomial is dominated by an exponential with base greater than 1, so $n^{\log_2 5}/2^n \to 0$.

The ranking from fastest to slowest is C, A, B. I would choose **algorithm** C, with running time $O(n^2 \log n)$.

2.9

I use the notation of Section 2.6 of [1]. For n a power of 2 and $\omega = e^{2\pi i/n}$, FFT(a, ω) returns $M_n(\omega)\, a$, the values $A(\omega^0), \ldots, A(\omega^{n-1})$ of the polynomial whose coefficient vector is $a = (a_0, \ldots, a_{n-1})$. It is computed recursively (Figure 2.9): split a into its even-indexed part and odd-indexed part, compute $s = \text{FFT}(\text{evens}, \omega^2)$ and $s' = \text{FFT}(\text{odds}, \omega^2)$, and set

$$r_j = s_j + \omega^j s'_j, \qquad r_{j+n/2} = s_j - \omega^j s'_j, \qquad 0 \le j < n/2,$$

with the base case $\omega = 1$ (that is, $n = 1$) returning a. Interpolation uses the inversion formula $M_n(\omega)^{-1} = \frac{1}{n} M_n(\omega^{-1})$, so the coefficients are $\frac{1}{n}\text{FFT}(\text{values}, \omega^{-1})$.

For $n = 2$ the recursion is one step: with $\omega = -1$, FFT$((u, v), -1) = (u + v,\ u - v)$. I use this for all the size-2 subproblems below.

Choice of n. In both parts the product has degree 3, so it has 4 coefficients and is uniquely determined by its values at any 4 distinct points (the Vandermonde matrix at distinct points is invertible). The smallest power of 2 that is at least 4 is $n = 4$. The 4th roots of unity $1, i, -1, -i$ are distinct, and $\omega = e^{2\pi i/4} = i$, $\omega^2 = -1$, $\omega^{-1} = -i$. Each input polynomial is padded with zero coefficients to length 4.

IDEA

Evaluate both polynomials at the four 4th roots of unity using the FFT, multiply the values pointwise to get the product's values at the same points, and run the FFT again with ω^{-1} (then divide by 4) to turn those values back into coefficients.

(a) $A(x) = 1 + x$ and $B(x) = 1 + x^2$, so $a = (1, 1, 0, 0)$ and $b = (1, 0, 1, 0)$.

FFT of a. Evens $(a_0, a_2) = (1, 0)$ give $s = (1, 1)$; odds $(a_1, a_3) = (1, 0)$ give $s' = (1, 1)$. Then

$$r_0 = s_0 + s'_0 = 2, \qquad r_2 = s_0 - s'_0 = 0,$$
$$r_1 = s_1 + i\, s'_1 = 1 + i, \qquad r_3 = s_1 - i\, s'_1 = 1 - i,$$

so FFT$(a, i) = (2,\ 1 + i,\ 0,\ 1 - i)$.

FFT of b. Evens $(1, 1)$ give $s = (2, 0)$; odds $(0, 0)$ give $s' = (0, 0)$. So FFT$(b, i) = (2,\ 0,\ 2,\ 0)$.

Componentwise product. $(2 \cdot 2,\ (1 + i) \cdot 0,\ 0 \cdot 2,\ (1 - i) \cdot 0) = (4, 0, 0, 0)$.

Inverse FFT. Compute FFT$((4, 0, 0, 0), -i)$: evens $(4, 0)$ give $s = (4, 4)$; odds $(0, 0)$ give $s' = (0, 0)$; so $r = (4, 4, 4, 4)$. Dividing by $n = 4$ gives the coefficients $(1, 1, 1, 1)$:

$$(x + 1)(x^2 + 1) = 1 + x + x^2 + x^3.$$

(b) $A(x) = 1 + x + 2x^2$ and $B(x) = 2 + 3x$, so $a = (1, 1, 2, 0)$ and $b = (2, 3, 0, 0)$.

FFT of a. Evens $(a_0, a_2) = (1, 2)$ give $s = (3, -1)$; odds $(a_1, a_3) = (1, 0)$ give $s' = (1, 1)$. Then

$$r_0 = 3 + 1 = 4, \qquad r_2 = 3 - 1 = 2,$$
$$r_1 = -1 + i \cdot 1 = -1 + i, \qquad r_3 = -1 - i \cdot 1 = -1 - i,$$

so $\mathrm{FFT}(a, i) = (4,\ -1 + i,\ 2,\ -1 - i)$.

FFT of b. Evens $(2, 0)$ give $s = (2, 2)$; odds $(3, 0)$ give $s' = (3, 3)$. Then

$$r_0 = 2 + 3 = 5, \qquad r_2 = 2 - 3 = -1,$$
$$r_1 = 2 + 3i, \qquad r_3 = 2 - 3i,$$

so $\mathrm{FFT}(b, i) = (5,\ 2 + 3i,\ -1,\ 2 - 3i)$.

Componentwise product.

$$\begin{aligned} 4 \cdot 5 &= 20, \\ (-1 + i)(2 + 3i) &= -2 - 3i + 2i + 3i^2 = -5 - i, \\ 2 \cdot (-1) &= -2, \\ (-1 - i)(2 - 3i) &= -2 + 3i - 2i + 3i^2 = -5 + i, \end{aligned}$$

giving the values $v = (20,\ -5 - i,\ -2,\ -5 + i)$ of $C = AB$ at $1, i, -1, -i$.

Inverse FFT. Compute $\mathrm{FFT}(v, -i)$. Evens $(v_0, v_2) = (20, -2)$ give $s = (18, 22)$. Odds $(v_1, v_3) = (-5 - i, -5 + i)$ give $s' = (-10, -2i)$. With $\omega^{-1} = -i$:

$$r_0 = s_0 + s'_0 = 18 - 10 = 8, \qquad r_2 = s_0 - s'_0 = 18 + 10 = 28,$$
$$r_1 = s_1 + (-i)s'_1 = 22 + (-i)(-2i) = 22 - 2 = 20, \qquad r_3 = s_1 - (-i)s'_1 = 22 + 2 = 24.$$

Dividing by 4 gives the coefficients $(2, 5, 7, 6)$:

$$(1 + x + 2x^2)(2 + 3x) = 2 + 5x + 7x^2 + 6x^3.$$

References

[1] S. Dasgupta, C. H. Papadimitriou, and U. V. Vazirani. *Algorithms*. Manuscript, July 18, 2006 (published by McGraw-Hill).

[2] Anthropic. *Claude* (model Claude Opus 5.5), accessed through Claude Code, October 2026. `https://claude.com/claude-code`

External resources and AI usage

The exercises are taken from Chapter 2 of [1].

I used Claude Opus 5.5, through Claude Code [2], to draft the solutions to all four exercises and to typeset this document in LaTeX. The trace of the divide-and-conquer multiplication in 2.1 was checked with a short Python implementation of Figure 2.1 of [1], and each FFT result in 2.9 was checked by multiplying the polynomials directly.